DIGT2107: Practice of Software Development

Project Iteration 3.3: Final Project Submission

OpenBid – Map-first Bidding Marketplace

Team 1: Tyler Mani Yanness Alaister

Due: March 29, 2026

Course: DIGT2107 – Winter Term 2026 | **Instructor:** Dr. May Haidar

1 Introduction

Project Name: OpenBid
Team Number: 1
Team Members: Tyler, Mani, Yanness, Alaister

Document Overview This document constitutes the final submission for Project Iteration 3.3 of the OpenBid platform. It consolidates all required deliverable components into a single professional artefact: the finalized Software Requirements Specification (SRS), the updated Software Design Document (SDD), comprehensive Testing Documentation, and a complete User Manual. The document reflects the system as it stands at the close of the project, incorporating all changes, refinements, and expansions made throughout each development iteration.

Project Summary OpenBid is a map-first bidding marketplace that connects contractors (job posters) with bidders (service providers). Users discover open jobs directly on an interactive Google Maps interface, submit competitive proposals, communicate in real time, and transact through a secure Stripe escrow payment system. The platform supports the complete job lifecycle – from discovery and bidding through award, payment, and post-job reviews – and includes an administrative moderation dashboard for platform governance.

Technology Stack

- **Frontend:** React 18, Vite, Google Maps JavaScript API, Stripe.js
- **Backend:** Node.js, Express, Socket.io
- **Database / Auth:** Firebase Firestore, Firebase Authentication
- **Payments / KYC:** Stripe Payments API, Stripe Identity
- **Two-Factor Authentication:** Duo Security (OIDC)
- **Testing:** Vitest + React Testing Library (client), Jest + Supertest (server)
- **Deployment:** Firebase Hosting, Google Cloud Run, Firebase Cloud Functions

2 Software Requirements Specification

2.1 Overview

This section presents the finalized Software Requirements Specification for the OpenBid platform. It reflects all functional and non-functional requirements as they stand at project completion, incorporating changes and additions made throughout the development process. Requirements are organized by feature domain and cross-referenced to the team members who led implementation of each area.

2.2 Functional Requirements

2.2.1 User Authentication and Account Management

ID	Requirement	Acceptance Criteria
FR-01	The system shall allow users to register with an email address and password.	A new user account is created and a verification email is sent in production mode.
FR-02	The system shall block login until the user's email address is verified (production mode).	An unverified user receives a clear error message on login.
FR-03	The system shall allow users to log in with email and password and receive a session token.	A valid Firebase ID token is returned and stored in the client session on successful login.
FR-04	The system shall support optional two-factor authentication via Duo Security.	When Duo credentials are configured, users are redirected to a Duo OIDC challenge after password verification before a session is established.
FR-05	The system shall allow users to reset their password via an emailed link.	A password reset link is sent and allows the user to set a new password within 24 hours.
FR-06	The system shall allow authenticated users to switch between the `contractor` and `bidder` roles at any time.	Switching role immediately updates visible UI elements and API permissions without requiring a logout.

2.2.2 Job Posting and Discovery

ID	Requirement	Acceptance Criteria
FR-07	Contractors shall be able to post a job with a title, description, budget, and a map-pinned location.	The job appears on the map immediately after submission and is visible to all bidders.

ID	Requirement	Acceptance Criteria
FR-08	Bidders shall be able to discover open jobs by browsing the interactive map.	Clicking a job pin opens a job detail view.
FR-09	Contractors shall be able to edit or delete their own job postings.	Edits are reflected immediately; deletion removes the job and all associated bids from the platform.

2.2.3 Bidding

ID	Requirement	Acceptance Criteria
FR-10	Bidders shall be able to submit a bid on any open job they did not post.	Submitting a bid on a job posted by the same user returns an error.
FR-11	Contractors shall be able to view all bids received on their jobs and award one bid as the winner.	Awarding a bid closes the job to further bidding and notifies the awarded bidder.
FR-12	Bidders shall be able to view and manage all bids they have submitted via the My Bids page.	Bid status (pending, awarded, rejected) is displayed and updated in real time.

2.2.4 Payments and KYC

ID	Requirement	Acceptance Criteria
FR-13	The system shall create a Stripe PaymentIntent in manual capture mode when a bid is awarded, placing funds into escrow.	A payment record is stored in Firestore with status `held` and a valid Stripe PaymentIntent ID.
FR-14	The contractor shall be able to release escrow funds to the awarded bidder upon job completion.	Calling the release endpoint captures the PaymentIntent and updates the Firestore payment status to `released`.
FR-15	The system shall require KYC identity verification via Stripe Identity before a payment release can be completed.	Releasing payment without a verified KYC status returns an error with a redirect URL to the Stripe Identity session.
FR-16	KYC verification status shall be persisted on the user's profile and checked on each release attempt.	A verified user is not prompted to complete KYC again unless their verification has expired.

ID	Requirement	Acceptance Criteria
FR-17	All payment operations shall function in prototype mode without real Stripe credentials, using a mock adapter.	The Fund, Release, and Refund buttons complete successfully in prototype mode with simulated responses.
FR-18	Payment action buttons shall be visible only to the job's contractor and only in states where the action is valid.	The Fund button appears only before escrow is funded; the Release and Refund buttons appear only after funding; no buttons appear after release or refund.

2.2.5 Real-Time Messaging

ID	Requirement	Acceptance Criteria
FR-19	Users shall be able to send and receive text messages in real time without a page refresh.	A message sent by one user appears in the other user's conversation window within 500 ms.
FR-20	Conversations shall be scoped to a specific job, keeping communication contextual.	Each conversation thread displays the associated job title and is inaccessible to users not party to that job.
FR-21	The message list shall automatically scroll to the most recent message when a new one arrives, with a scroll guard that respects the user's current position.	Auto-scroll triggers only when the user is within 100 px of the bottom; reading older messages is not interrupted.
FR-22	Messages sent by the current user shall appear on the right; messages from others on the left.	CSS classes are applied based on sender UID comparison, providing clear visual distinction.
FR-23	Message history shall be persisted in Firestore and available on reconnect.	Reopening the messenger after a disconnect restores all previous messages in chronological order.
FR-24	Admin users shall be forbidden from accessing the messaging system.	A request to any messaging endpoint from an admin-role token returns HTTP 403.

2.2.6 Profile, Portfolio, and Reviews

ID	Requirement	Acceptance Criteria
FR-25	Users shall be able to upload and update a profile avatar image.	The new avatar is stored in Firebase Storage and displayed on the user's profile within one page reload.
FR-26	Users shall be able to view any other user's public profile, including bio, avatar, portfolio, and reviews.	The `/profile/:userId` route renders the target user's public information without requiring the viewer to be logged in.
FR-27	Users shall be able to add portfolio items, each consisting of an image, title, and description.	A submitted portfolio item appears on the user's profile immediately after submission.
FR-28	After a completed job, the contractor and the awarded bidder shall each be able to leave one review for the other.	Submitting a second review for the same job and person returns HTTP 409.
FR-29	Reviews shall include a star rating of 1 to 5 and a text comment, and the reviewee's average rating shall be recalculated after each new review.	The displayed average rating updates immediately after a review is submitted.

2.2.7 Admin Dashboard

ID	Requirement	Acceptance Criteria
FR-30	The system shall provide an Admin Dashboard accessible only to users with the admin flag.	Navigating to `/admin` without admin privileges redirects to the home page.
FR-31	The Admin Dashboard shall display searchable, paginated lists of all users, jobs, and bids across three separate tabs.	Switching tabs updates the displayed content without a full page reload.
FR-32	Admins shall be able to edit any user, job, or bid via an inline editor modal that pre-populates all current values.	Changes submitted through the modal are immediately reflected in the table without a full reload.
FR-33	Admins shall be able to delete any user, job, or bid after confirming a deletion dialog.	The record is removed from the table within 300 ms of confirmation.
FR-34	The system shall prevent admins from deleting their own account through the dashboard.	Attempting to delete the currently authenticated admin account returns an error.

ID	Requirement	Acceptance Criteria
FR-35	Client-side route guards shall prevent non-admin and unauthenticated users from accessing admin routes.	The `AdminRoute` wrapper component redirects these users to the home or login page.

2.3 Non-Functional Requirements

Category	Requirement
Security	All routes that create, modify, or delete data require a valid Firebase ID token. Stripe operations use server-side secret keys exclusively.
Prototype Mode	The full application shall run without any external service credentials by switching all adapters to their mock implementations via the `PROTOTYPE=TRUE` environment flag.
Testability	Every server-side route module shall have a corresponding mock-mode test suite. Integration tests shall use the Firebase local emulator suite.
Usability	The UI shall be responsive and fully functional on screens 360 px wide and above.
Data Integrity	Destructive admin operations require explicit user confirmation dialogs before execution.
Availability	Prototype mode must allow full application functionality for demonstration and development without any live service dependency.

3 Software Design Document

3.1 Architectural Overview

OpenBid follows a layered client-server architecture with a strict separation between the React single-page application, the Express REST and Socket.io server, and the Firebase and Stripe backend services. The central architectural principle is the **adapter pattern**: every external dependency (database, authentication, payments, KYC, storage) has two interchangeable implementations – a `*.real.js` file that communicates with live services and a `*.mock.js` file used in prototype mode and testing. The active implementation is selected at startup from the `PROTOTYPE` environment variable, enabling the entire system to operate without external credentials.

React Client (Vite, React 18) Pages Components Hooks Services
↓↑ HTTP REST ↓↑ WebSocket (Socket.io)
Express Server Routes Middleware (RBAC) Socket.io
↓↑ Adapter Layer
Firebase Firestore & Auth **Stripe Payments & Identity** **Duo Security**

Figure 1: High-Level System Architecture

3.2 Component Diagram

Figure 2 presents the system's major components and their dependency relationships. The adapter layer is the central architectural seam: every route module depends on adapter interfaces rather than concrete implementations, enabling the prototype/production swap.

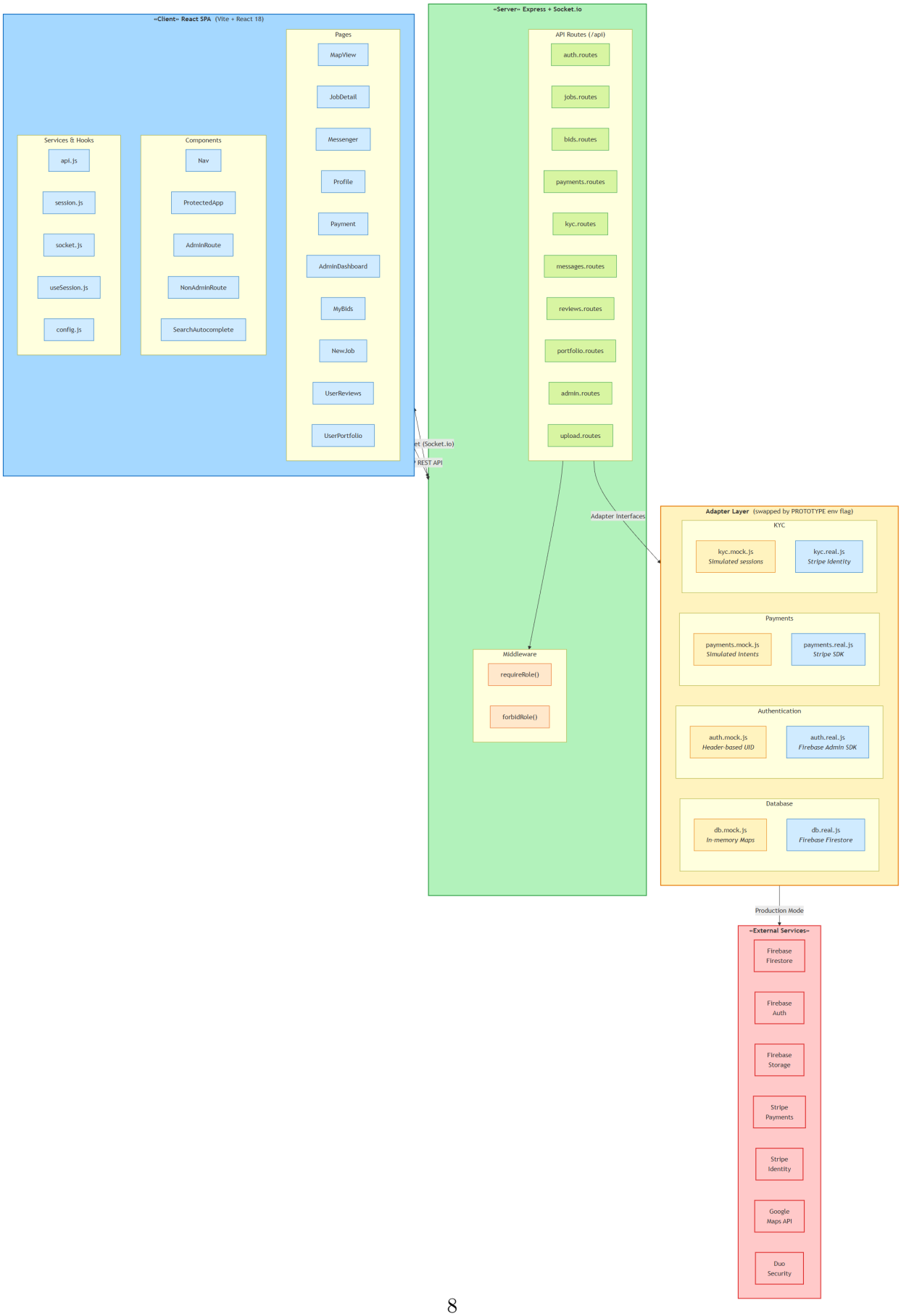


Figure 2: Component Diagram – dependencies flow top-to-bottom through the adapter layer

Adapter Interface Methods Each adapter exposes a consistent interface regardless of its backing implementation. Table 9 summarises the key method groups.

Adapter	Sub-module	Methods
db	user	create, get, update, delete, findByEmail, list, upsert
	job	create, get, update, delete, list
	bid	create, get, update, delete, listByJob, listByUser, deleteByJob, list
	conversations	create, get, find, listByUser, markRead, hide, unhide, delete
	messages	create, list
	review	create, get, update, delete, listByReviewed, listByJob, appendPhotos
	portfolio	create, get, update, delete, listByUser, appendPhotos
auth	–	signIn(email, password), verify(req)
payments	–	createPaymentIntent, capture, refund, cancel, getStatus
kyc	–	verification(uid), status(uid)

3.3 Class Diagram – Data Model

Figure 3 presents the system's core data entities and their relationships as stored in Firebase Firestore.

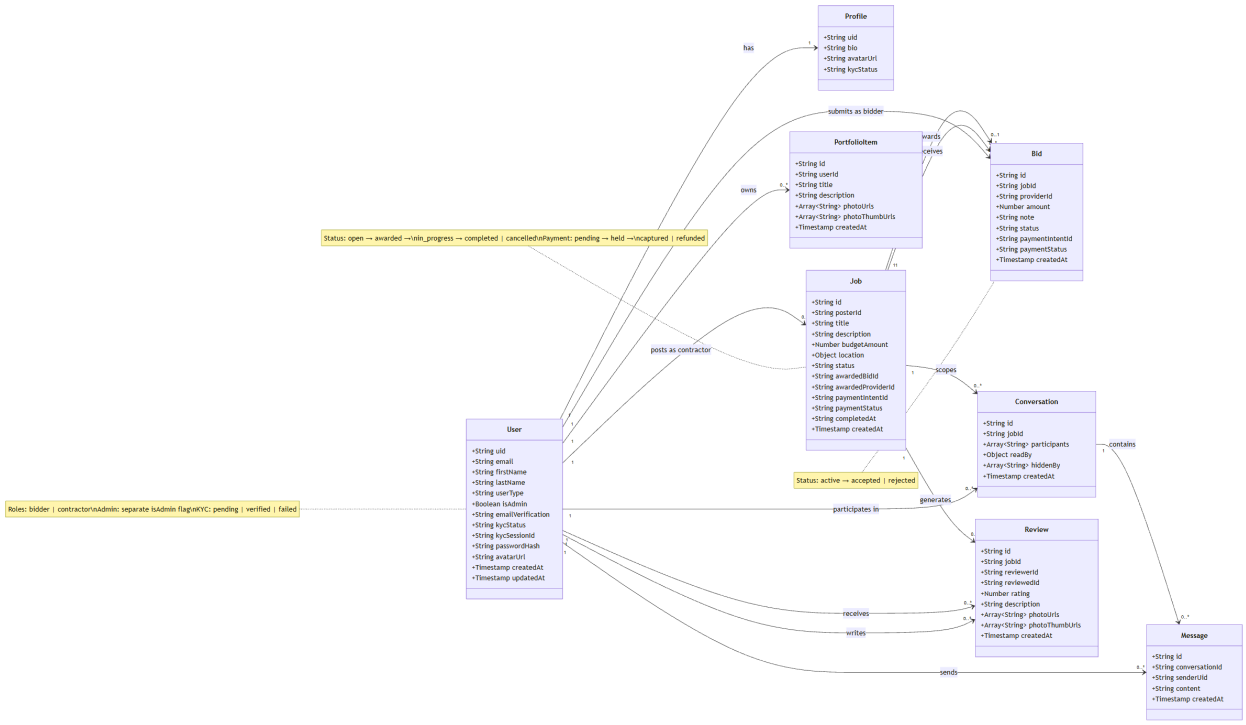
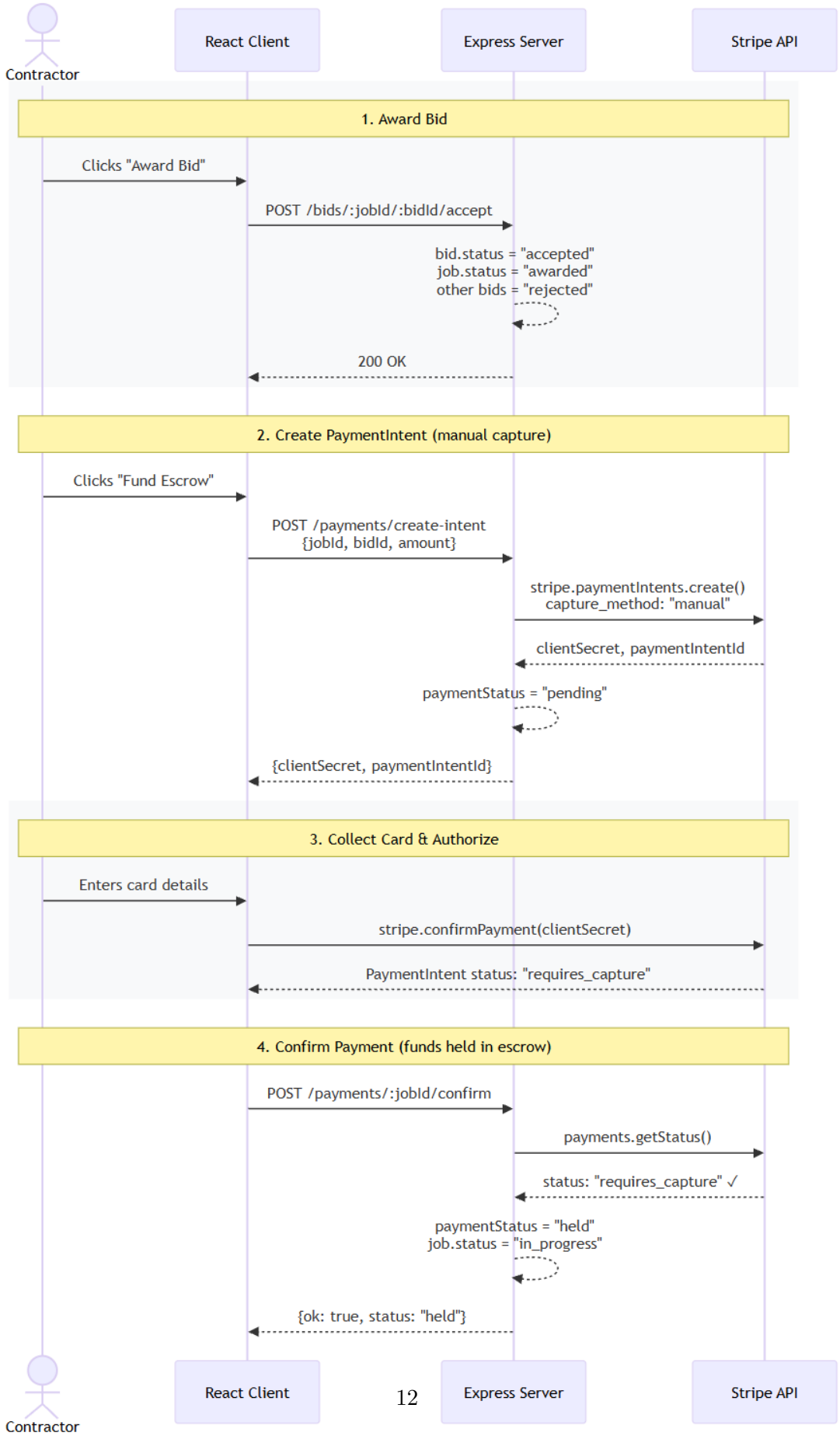


Figure 3: Class Diagram – Core data entities and relationships

3.4 Sequence Diagram – Payment Escrow Flow

Figures 4 and 5 trace the full escrow lifecycle from bid award through fund capture or refund.


Contractor
React Client
Express Server
Stripe API
1. Award Bid
Clicks "Award Bid"
POST /bids/:jobId/:bidId/accept
bid.status = "accepted"
job.status = "awarded"
other bids = "rejected"
200 OK
2. Create PaymentIntent (manual capture)
Clicks "Fund Escrow"
POST /payments/create-intent
{jobId, bidId, amount}
stripe.paymentIntents.create()
capture_method: "manual"
clientSecret, paymentIntentId
paymentStatus = "pending"
{clientSecret, paymentIntentId}
3. Collect Card & Authorize
Enters card details
stripe.confirmPayment(clientSecret)
PaymentIntent status: "requires_capture"
4. Confirm Payment (funds held in escrow)
POST /payments/:jobId/confirm
payments.getStatus()
status: "requires_capture" ✓
paymentStatus = "held"
job.status = "in_progress"
{ok: true, status: "held"}
Contractor
React Client
Express Server
Stripe API

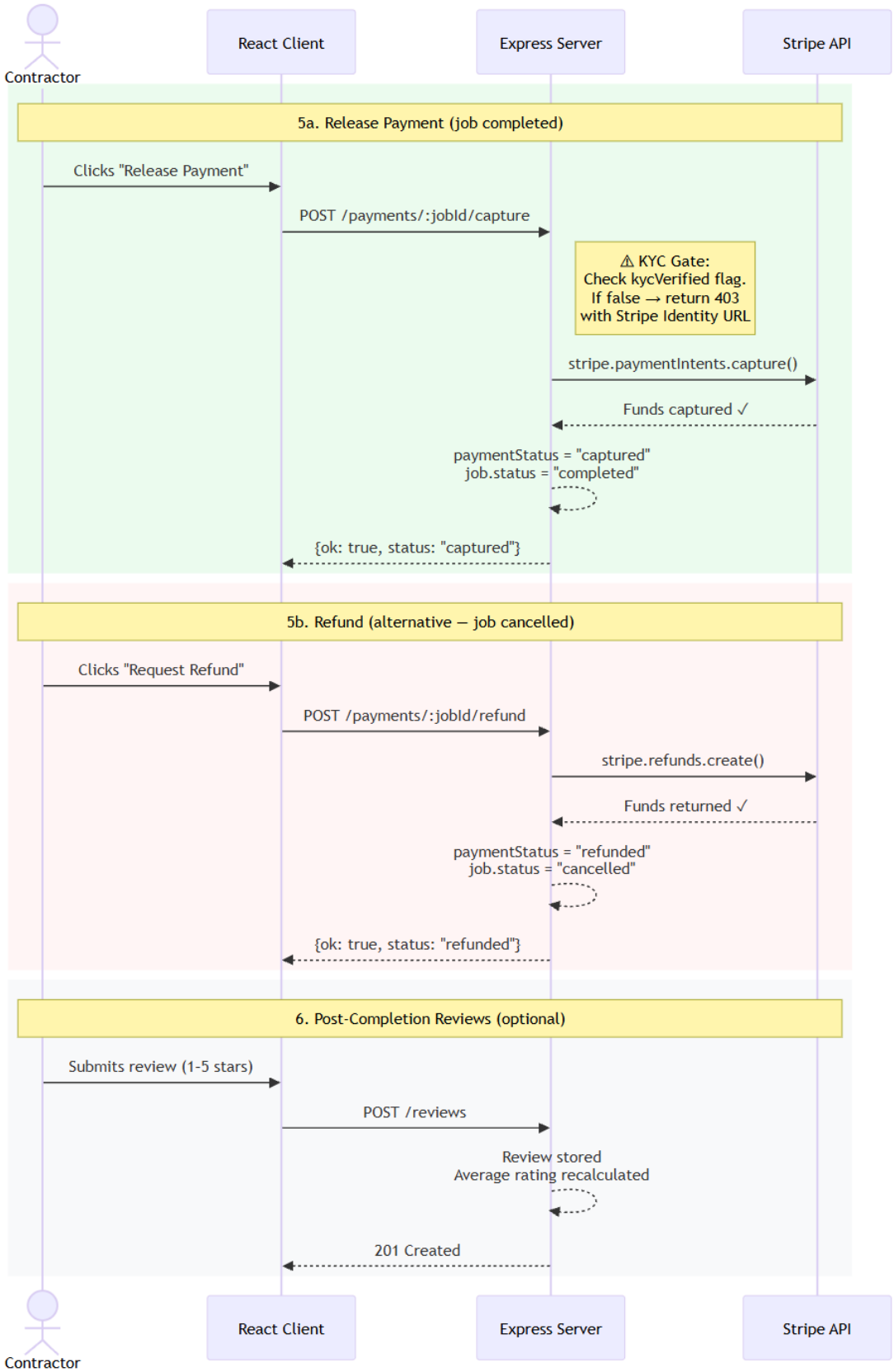
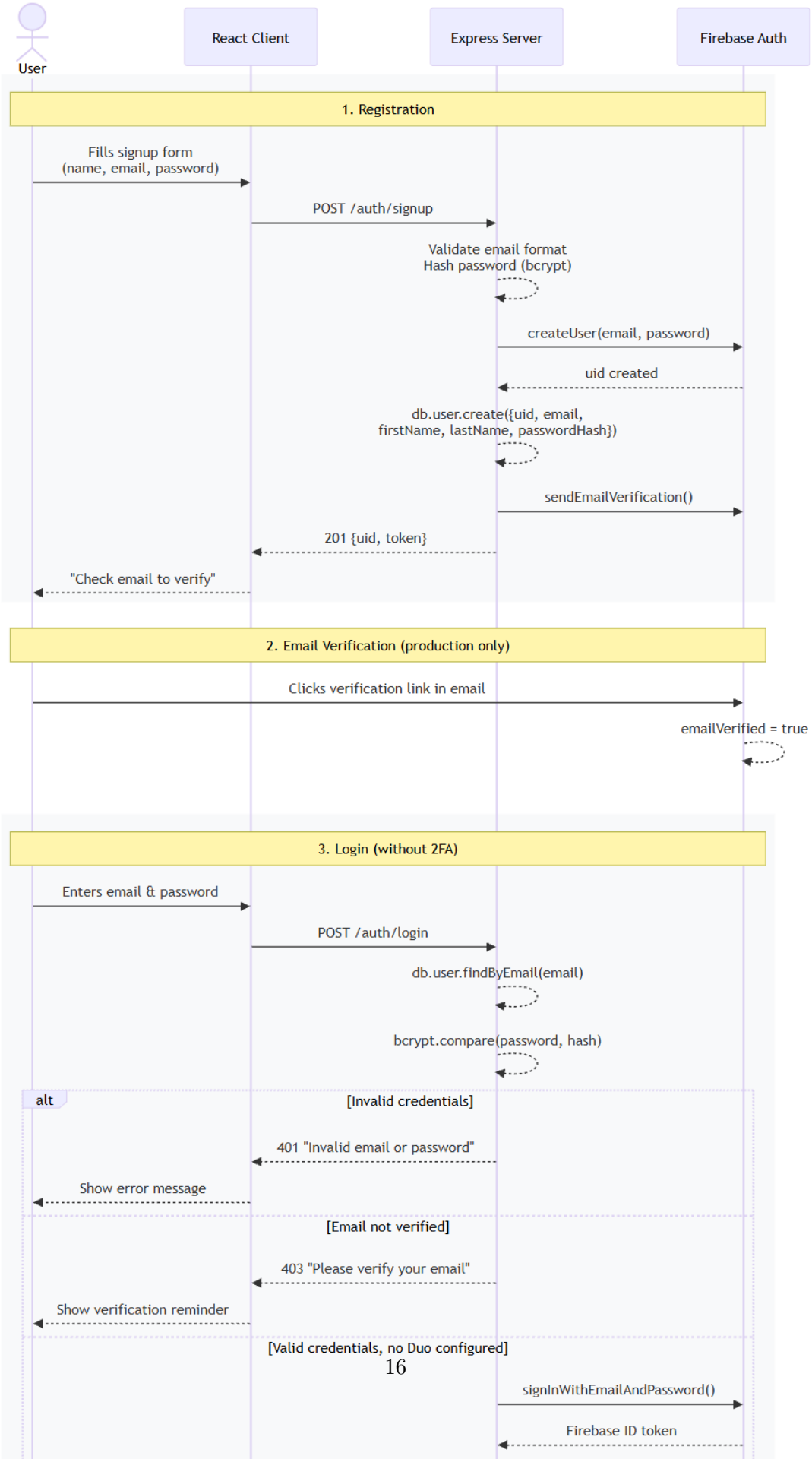


Figure 5: Payment Escrow Sequence – Steps 5–6: Release/Refund and Reviews

KYC Gate (within step 5a) Before executing a capture, the server retrieves the contractor's `kycVerified` flag from Firestore. If `false`, the server calls `kyc.verification(uid)` to create a Stripe Identity session and returns the session URL with HTTP 403. The contractor completes identity verification on the Stripe-hosted page; a webhook event updates `kycVerified = true`. The next capture attempt proceeds normally.

3.5 Sequence Diagram – Authentication and Two-Factor Flow

Figures 6 and 7 trace the authentication flow including optional Duo Security two-factor authentication.


User
React Client
Express Server
Firebase Auth
1. Registration
Fills signup form
(name, email, password)
POST /auth/signup
Validate email format
Hash password (bcrypt)
createUser(email, password)
uid created
db.user.create({uid, email,
firstName, lastName, passwordHash})
sendEmailVerification()
201 {uid, token}
"Check email to verify"
2. Email Verification (production only)
Clicks verification link in email
emailVerified = true
3. Login (without 2FA)
Enters email & password
POST /auth/login
db.user.findByEmail(email)
bcrypt.compare(password, hash)
alt
[Invalid credentials]
401 "Invalid email or password"
Show error message
[Email not verified]
403 "Please verify your email"
Show verification reminder
[Valid credentials, no Duo configured]
signInWithEmailAndPassword()
Firebase ID token

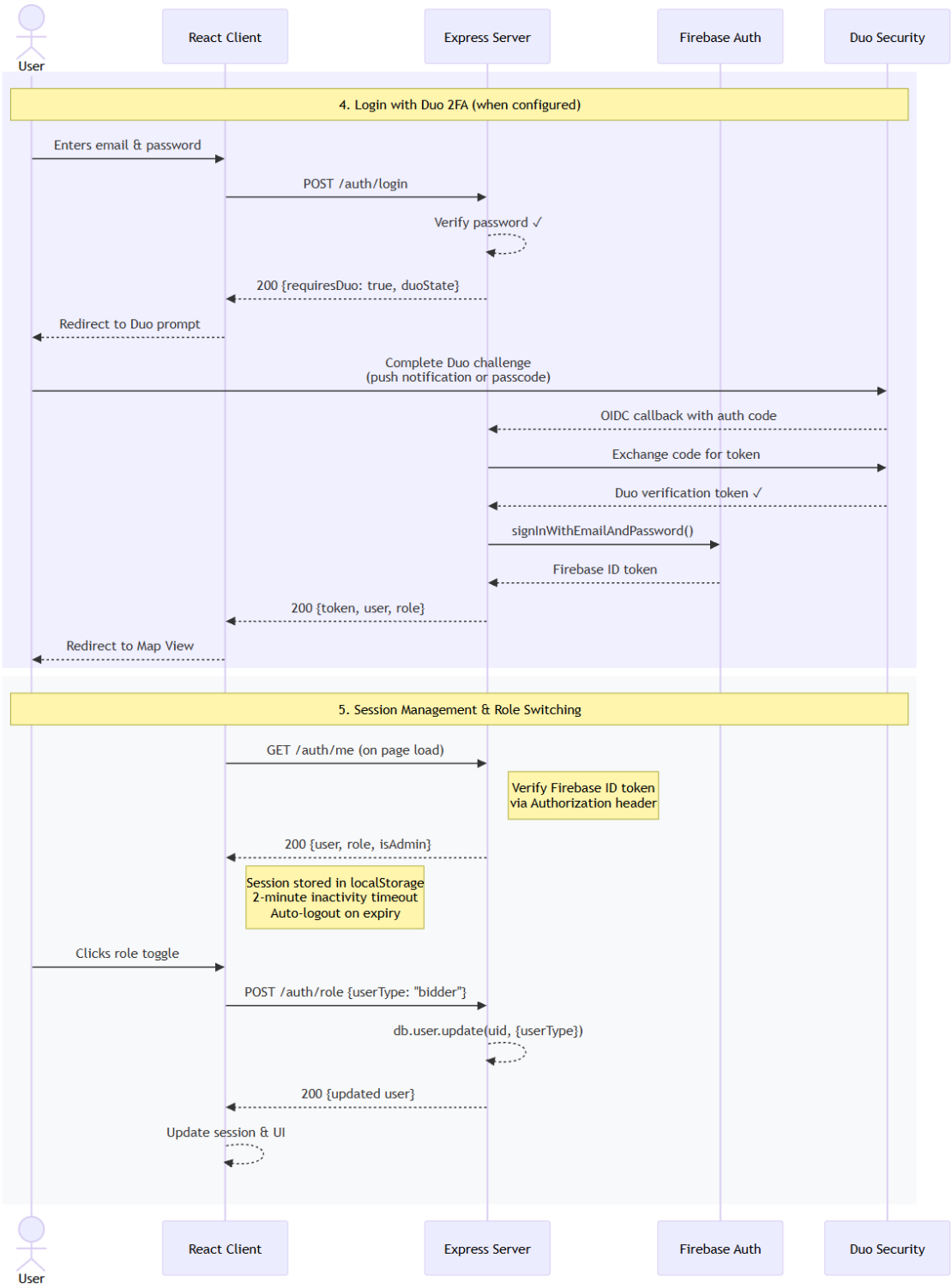


Figure 7: Authentication Sequence – Steps 4–5: Duo 2FA and Session Management

3.6 Activity Diagram – Job Lifecycle

Figure 8 models the complete lifecycle of a job from creation through completion or cancellation, including the payment and review sub-flows.

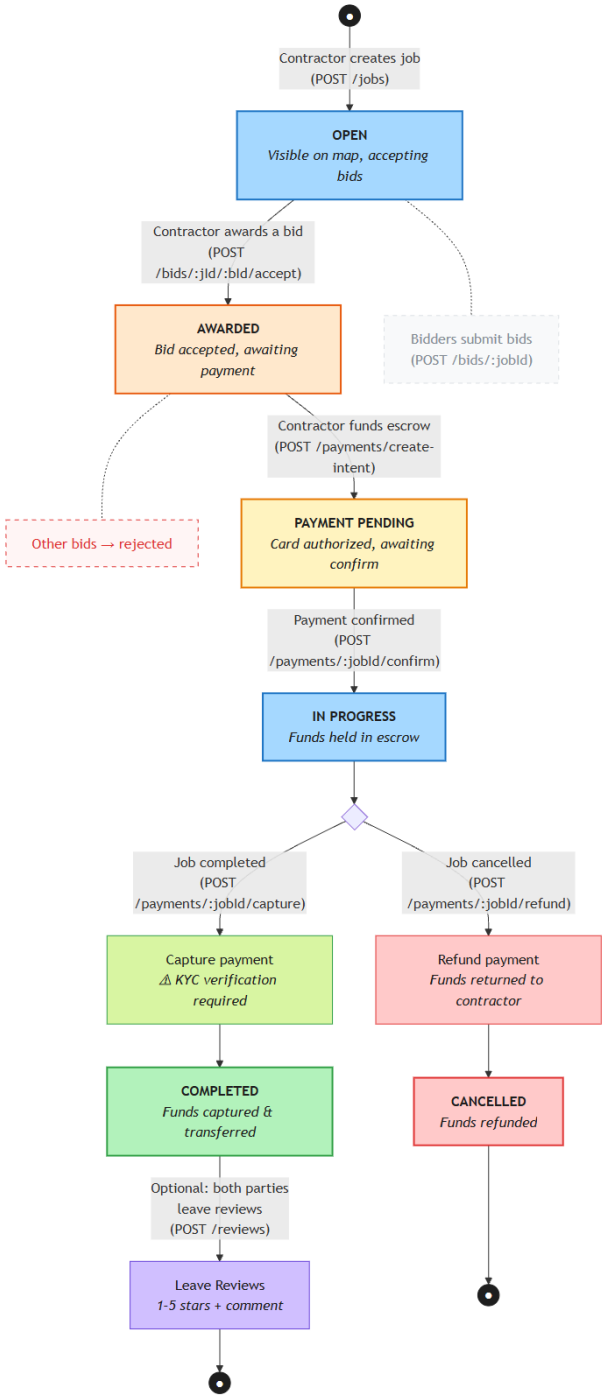


Figure 8: Activity Diagram – Job Lifecycle with Payment and Review Sub-flows

State Transitions Summary

From	Trigger	To
`open`	Contractor awards bid	`awarded`
`awarded`	Payment confirmed	`in_progress`
`in_progress`	Contractor captures payment	`completed`
`in_progress`	Contractor requests refund	`cancelled`

3.7 Role and Access Control Model

Every authenticated user holds one of two operational roles – `contractor` or `bidder` – and may switch freely between them. A separate boolean `isAdmin` flag grants elevated platform privileges. Role is stored on the Firestore user document and embedded in the session context so that both client-side route guards and server-side middleware can enforce it independently.

Server-Side Enforcement The `requireRole.js` middleware provides two exported functions: `requireRole(...allowedRoles)`, which permits only listed roles through, and `forbidRole(...blockedRoles)`, which blocks listed roles regardless of other permissions. These are composed on individual routes. The messaging system, for example, uses `forbidRole("admin")` to prevent administrators from participating in user conversations.

Client-Side Enforcement Two React wrapper components enforce access at the routing layer. `AdminRoute` redirects any user who is unauthenticated or lacks the admin flag to the home page. `NonAdminRoute` can redirect admin users away from standard user flows where appropriate. Both components read from the shared `useAuth` context hook, which is populated synchronously from cached session state to eliminate any flash of protected content.

3.8 Authentication and Session Management

Firebase Authentication issues ID tokens on successful login. The Express server validates these tokens on every protected request using the Firebase Admin SDK. The validated UID is attached to the request object and used for all subsequent authorization checks and database queries.

When Duo Security is enabled, the authentication flow includes an additional OIDC redirect after password verification. The server initiates the Duo authorization request, the user completes the Duo push or passcode challenge, and Duo's callback exchanges an authorization code for a confirmation token. Only after this exchange is the Firebase session fully established.

3.9 Payment System Design

The payment system implements a three-state escrow model using Stripe PaymentIntents in manual capture mode.

State Machine

$$\texttt{unfunded} \xrightarrow[\text{contractor funds}]{} \texttt{held} \xrightarrow[\text{job complete}]{} \texttt{released}$$

$$\texttt{held} \xrightarrow[\text{job cancelled}]{} \texttt{refunded}$$

State transitions are guarded server-side: only the contractor who created the job may trigger a release or refund, enforced by comparing the authenticated UID against the job's `contractorId` field before any Stripe API call is made.

KYC Gate Before executing a payment release, the server checks the `kycVerified` flag on the user's Firestore document. If unverified, the server creates a Stripe Identity `VerificationSession` and returns the session URL. When the user completes the identity flow, Stripe sends a signed webhook event that the server uses to update `kycVerified = true`. The release can then proceed on the user's next attempt.

Client-Side Button Visibility Payment button visibility is computed from a single `paymentStatus` value fetched from the server when the Job Detail page loads. A mapping function translates the status to the set of currently valid actions, keeping the JSX declarative and independently testable.

3.10 Real-Time Messaging Design

The messaging system uses Socket.io for bidirectional real-time communication. Conversations are keyed by a canonical room ID derived from the job ID and the two participant UIDs (lexicographically sorted to ensure a single unique key per pair). When a client connects, it emits a `join` event; the server joins the socket to the appropriate room. All subsequent `sendMessage` events are emitted to the full room (including the sender's socket), ensuring both participants see messages without duplication.

Messages are simultaneously persisted to Firestore, providing a durable history that is loaded on reconnection. The client-side scroll guard tracks the user's scroll position and only auto-scrolls to new messages when the user is already within 100 px of the bottom of the conversation, preserving reading position when browsing history.

3.11 Admin Dashboard Design

The Admin Dashboard is a tabbed single-page component. Each tab – Users, Jobs, Bids – is an independent child component that fetches its data on mount and maintains its own local search and pagination state. The active tab is stored in the URL query string (`?tab=users`) so that the browser's history stack preserves tab state across navigation.

Entity editing is handled by a single reusable `EditorModal` component that accepts the target entity, a field schema descriptor, and save/close callbacks. The field schema drives dynamic form generation, allowing the same modal to serve all three entity types without duplicated layout code.

3.12 Profile, Portfolio, and Reviews Design

User profiles aggregate data from three Firestore sources: the core user document (bio, avatar URL, KYC status), a `portfolio` sub-collection, and a `reviews` collection filtered by `revieweeId`. The profile page fetches all three in parallel and displays a computed average star rating derived from the review collection.

Portfolio image uploads and avatar uploads both follow a two-step flow: the client POSTs the file to the `/upload` route, the server validates the MIME type and file size via Multer middleware, stores the file in Firebase Storage, and returns the public URL. The client then PATCHes the relevant record with the new URL.

3.13 System Component Summary

Component	Type	Technology	Responsibility
`AdminDashboard.jsx`	Page	React	Tabbed admin interface: users, jobs, bids
`AdminRoute.jsx`	Component	React	Client-side admin route guard
`NonAdminRoute.jsx`	Component	React	Client-side non-admin route guard
`Messenger.jsx`	Page	React, Socket.io	Real-time chat UI with scroll guard
`Payment.jsx`	Page	React, Stripe.js	Escrow funding via Stripe Elements
`Profile.jsx`	Page	React	User profile with avatar, portfolio, reviews
`UserPortfolio.jsx`	Page	React	Portfolio management and display
`UserReviews.jsx`	Page	React	Review submission and listing
`MapView.jsx`	Component	Google Maps API	Job pin display and interaction
`admin.routes.js`	Router	Express	Admin CRUD for users, jobs, bids
`payments.routes.js`	Router	Express, Stripe	Escrow create, release, refund
`kyc.routes.js`	Router	Express, Stripe Identity	KYC session creation and webhook
`messages.routes.js`	Router	Express, Socket.io	Conversation and message management
`auth.routes.js`	Router	Express, Firebase	Login, signup, Duo 2FA
`reviews.routes.js`	Router	Express	Review CRUD and aggregation
`portfolio.routes.js`	Router	Express	Portfolio item CRUD
`upload.routes.js`	Router	Express, Multer	Avatar and photo upload handling
`requireRole.js`	Middleware	Express	Server-side RBAC enforcement
`payments.real.js`	Adapter	Stripe SDK	Production payment integration
`payments.mock.js`	Adapter	In-memory	Prototype mode payment simulation
`kyc.real.js`	Adapter	Stripe Identity SDK	Production KYC integration
`kyc.mock.js`	Adapter	In-memory	Prototype mode KYC simulation
`db.real.js`	Adapter	Firebase Firestore	Production data persistence
`db.mock.js`	Adapter	In-memory	Prototype mode data simulation
`auth.real.js`	Adapter	Firebase Auth	Production authentication
`auth.mock.js`	Adapter	In-memory	Prototype mode auth simulation

4 Testing Documentation

4.1 Testing Methodology

The project employs a two-layer testing strategy designed to provide confidence at both the unit and integration levels while maintaining fast, reliable test execution without mandatory dependencies on live external services.

Server-Side Testing Server-side tests are written with Jest and Supertest. Each route module has a **mock-mode** test suite that substitutes all external adapters (database, authentication, payments, KYC) with in-memory implementations. Where the system behavior depends on real Firebase or Stripe interactions, separate **integration tests** run against the Firebase local emulator suite in CI.

Client-Side Testing Client-side tests are written with Vitest and React Testing Library. Components are rendered in a simulated DOM with mocked API services and context providers, testing user-facing behaviour without a running backend.

Smoke Tests A dedicated smoke test suite validates critical-path behaviour when the application runs with `PROTOTYPE=FALSE`, confirming that the real Firebase and Stripe adapters initialise and respond correctly. These tests are designed to run quickly as a pre-deployment gate.

Test Execution

```
# Run server-side tests
cd server && npm test

# Run client-side tests
cd client && npm test
```

4.2 Test Suites and Coverage

4.2.1 Authentication Tests

Files `auth.integration.real.routes.test.js`, `auth.duo.test.js`

Coverage

- Sign-up with valid credentials creates a user and returns a Firebase ID token.
- Login with correct credentials returns a valid session token.
- Login with an incorrect password returns HTTP 401.
- All protected routes return HTTP 401 when called without a token.
- All protected routes return HTTP 401 when called with a malformed or expired token.
- Duo 2FA callback: a valid authorization code completes the session; an invalid or replayed code returns HTTP 403.
- Password reset: requesting a reset for a registered email dispatches a reset link; requesting for an unknown email returns a safe, non-enumerable response.

4.2.2 Payment System Tests

Files `payments.routes.test.js`

Coverage The payment test suite contains 30 test cases covering the full escrow lifecycle:

- **Escrow creation:** Returns HTTP 201 with a `clientSecret` for a valid bid award; returns HTTP 400 for missing required fields; returns HTTP 409 when a payment already exists for the job.
- **Payment confirmation:** Transitions job status to `in_progress`; prevents confirmation by non-contractors.
- **Payment release:** Captures the Stripe PaymentIntent and updates status to `released` for a KYC-verified contractor; returns HTTP 403 with a `kycUrl` for unverified users; returns HTTP 403 for non-contractors.
- **Refund:** Cancels or refunds the PaymentIntent and sets status to `refunded`; returns HTTP 404 for unknown payment IDs.
- **Status query:** Both the contractor and awarded bidder can view payment status; all other users receive HTTP 403.
- **Webhook handling:** A correctly-signed `payment_intent.succeeded` event updates the Firestore payment record; unsigned or tampered webhooks return HTTP 400.

4.2.3 KYC Tests

Files `kyc.mock.routes.test.js`, `kyc.real.routes.test.js`

Coverage

- Mock mode: `POST /kyc/session` returns a synthetic session URL; the webhook endpoint sets `kycVerified = true` on the mock user document.
- Real mode (emulator): session creation returns a well-formed Stripe Identity URL; the webhook handler correctly processes a test verification event.
- Unauthenticated requests to KYC endpoints return HTTP 401.

4.2.4 Jobs and Bids Integration Tests

Files `jobs.integration.real.routes.test.js`, `bids.integration.real.routes.test.js`

Coverage

- Full lifecycle test: create job → submit bid → award bid → verify payment record is created in Firestore.
- Bidding on a job you own returns HTTP 403.
- Awarding a bid on another user's job returns HTTP 403.
- Listing jobs returns only open jobs by default; awarded and closed jobs appear with the correct filter.

- Editing a job you do not own returns HTTP 403.
- Deleting a job removes all associated bids.

4.2.5 Admin Route Tests

Files `admin.mock.routes.test.js`, `admin.real.routes.test.js`

Coverage – 19 test cases

- **GET /admin/users, /admin/jobs, /admin/bids:** Returns complete lists; returns HTTP 401 without authentication; returns HTTP 403 for non-admin authenticated users.
- **PATCH /admin/users/:id:** Returns HTTP 200 with the updated entity; returns HTTP 400 for an invalid email address format.
- **DELETE /admin/users/:id:** Returns HTTP 204; a subsequent GET confirms the record is absent; attempting to delete the authenticated admin's own account returns HTTP 400.
- **Response formatting:** User listings exclude sensitive fields such as password hashes; bid listings are returned in descending chronological order.
- Real-mode tests confirm that mutations are actually applied to the Firebase emulator, not just mocked.

4.2.6 Messaging Tests

Files `messages.mock.routes.test.js`, `messages.integration.real.routes.test.js`

Coverage – 29 unit tests (mock) + integration tests

- **Conversation creation:** A conversation is created when a bid is placed; duplicate creation for the same pair is idempotent.
- **Listing conversations:** Returns only conversations the authenticated user participates in; unauthenticated requests return HTTP 401.
- **Message sending:** Returns HTTP 200 with the stored message object; HTTP 400 for an empty message body; HTTP 403 for non-participant senders.
- **Message history:** Retrieved in chronological order; read markers update correctly; non-participants receive HTTP 403 or 404.
- **Hide / Unhide / Delete:** Hide removes the conversation from the default listing; unhide restores it; delete permanently removes it; only participants may perform these actions.
- **Admin restriction:** Any messaging endpoint accessed with an admin token returns HTTP 403.
- **Integration:** Two emulator users exchange messages; history is verified on reconnect; messages in one room do not appear in another room.

4.2.7 Reviews Tests

Files `reviews.routes.test.js`

Coverage – 7 test cases

- A review can only be submitted after the job has been awarded, and only for the awarded provider – attempts before award or for the wrong user are rejected.
- Users cannot review themselves; users not party to the job cannot submit a review.
- Duplicate review submissions (same reviewer, same job, same reviewee) return HTTP 409.
- Review photos upload correctly and appear in the user's review list with thumbnail links.
- Only the review author can edit the review; the rating and text can be updated independently; clearing the description does not invalidate the review.
- Review listing returns records in reverse-chronological order; the computed average rating reflects all submitted reviews.

4.2.8 Upload Tests

Files `upload.avatar.routes.test.js`

Coverage

- A valid JPEG or PNG upload returns HTTP 200 with a public storage URL in the response body.
- Uploading a file with an unsupported MIME type (e.g. `application/pdf`) returns HTTP 400.
- A request with no file attached returns HTTP 400 with a descriptive error message.
- A file exceeding the configured size limit returns HTTP 413.

4.2.9 Client-Side Component Tests

Admin Dashboard Tests `(AdminDashboard.test.jsx)`

- The component renders without crashing when supplied with mock user, job, and bid data.
- Clicking a tab header updates the visible content pane to the correct section.
- Opening the editor for a record pre-populates the email field with the record's current value.
- Clicking the delete button surfaces a confirmation dialog before dispatching any delete action.

Navigation and Session Tests `(nav.test.jsx, session.test.js)`

- The `Nav` component displays the correct role label for the user's active role.
- The admin dashboard link is visible only when `isAdmin` is `true`.
- Clicking the role switch button invokes the role change handler exactly once.
- The `useSession` hook reads the active role from `localStorage` on mount; calling `switchRole` writes the new role and updates the context; clearing storage and re-mounting defaults to the server-supplied role.

Messenger and Socket Tests `(Messenger.test.jsx, socket.test.js)`

- The Messenger page renders an empty-state placeholder when no conversation is selected.
- Messages from the current user carry the `.message-sent` CSS class; messages from others carry `.message-received`.
- When a new message arrives and `nearBottom` is `true`, the container scrolls to the bottom; when `nearBottom` is `false`, the scroll position is unchanged.
- The socket service returns the same singleton instance across multiple calls to `getSocket`.
- `connectSocket` and `disconnectSocket` correctly manage the connection lifecycle.
- Event subscriptions via `onNewMessage` can be unsubscribed without leaking listeners.

Profile Tests `(Profile.test.jsx)`

- The Profile component renders the user's name and bio from mock data.
- Portfolio items are displayed as a grid; an empty portfolio shows a placeholder message.
- The average star rating is computed and rendered correctly from mock review data.
- The review submission form is not rendered when the viewer is viewing their own profile.

Map Utility Tests `(mapHelpers.test.js, locationHelpers.test.js)`

- `formatCoordinate` rounds latitude and longitude values to four decimal places.
- `isWithinBounds` correctly classifies coordinates inside and outside a given bounding box, including coordinates exactly on the boundary.
- `calculateDistance` returns the correct Haversine distance within a 1% tolerance for known city coordinate pairs, including the edge case of zero distance.
- `parseGeoPoint` converts a Firestore GeoPoint into a plain `{lat, lng}` object.

4.3 Test Results Summary

Test Suite	Tests	Status	Notes
auth routes (real)	8	Pass	Firebase emulator
auth.duo	4	Pass	Mock Duo OIDC
payments routes	30	Pass	Mock adapter
kyc routes (mock)	6	Pass	Mock adapter
kyc routes (real)	4	Pass	Firebase emulator
jobs integration (real)	9	Pass	Firebase emulator
bids integration (real)	7	Pass	Firebase emulator
admin routes (mock)	11	Pass	Mock adapter
admin routes (real)	8	Pass	Firebase emulator
messages routes (mock)	29	Pass	Mock adapter
messages integration (real)	6	Pass	Firebase emulator
reviews routes	7	Pass	Mock adapter
upload avatar routes	5	Pass	Mock storage

Test Suite	Tests	Status	Notes
AdminDashboard component	4	Pass	Vitest + RTL
nav component	4	Pass	Vitest + RTL
session hook	3	Pass	Vitest
Messenger component	6	Pass	Vitest + RTL
socket service	7	Pass	Vitest
Profile component	4	Pass	Vitest + RTL
mapHelpers	5	Pass	Vitest
locationHelpers	3	Pass	Vitest
Total	**170**	**All Pass**	

4.4 Code Coverage Reports

Code coverage was collected using V8 (client, via Vitest) and Istanbul (server, via Jest). The tables below report statement, branch, function, and line coverage percentages for each source module.

4.4.1 Server-Side Coverage

File / Directory	% Stmts	% Branch	% Funcs	% Lines
All files	**50.59**	**38.92**	**53.80**	**53.72**
src/				
config.js	100	72.22	100	100
src/routes/				
auth.routes.js	64.03	52.70	100	64.28
jobs.routes.js	64.80	60.86	77.77	71.81
bids.routes.js	42.14	29.07	40.00	44.96
payments.routes.js	79.71	80.45	100	78.90
kyc.routes.js	87.09	92.85	100	89.28
messages.routes.js	90.66	85.18	88.23	91.66
reviews.routes.js	74.00	51.92	90.00	81.81
upload.routes.js	73.62	45.76	90.00	78.82
admin.routes.js	19.14	13.80	31.42	21.16
src/adapters/				
db.real.js	51.32	26.57	57.50	57.91
kyc.real.js	44.18	25.00	100	42.85
auth.mock.js	10.00	0	0	11.11
auth.real.js	6.66	0	0	7.69
payments.mock.js	16.66	100	0	16.66
kyc.mock.js	25.00	0	0	25.00
db.mock.js	1.58	0	0	1.73
src/lib/				
duoState.js	33.33	0	33.33	36.36
firebase.js	5.26	0	0	5.55
firebaseIdentity.js	13.51	0	11.11	14.28
src/___tests___/				
testUtils.js	75.70	70.33	80.00	76.94

12 test suites passed, 121 tests passed (2 skipped).

Analysis Route modules that are the primary targets of mock and integration test suites show the highest coverage: `messages.routes.js` (91.66% lines), `kyc.routes.js` (89.28%), and `payments.routes.js` (78.90%). Adapter files such as `db.mock.js` and `auth.mock.js` show low statement coverage because Jest's instrumentation records them as loaded source files, but their methods are exercised indirectly through the route tests rather than through direct unit tests. The `admin.routes.js` file has lower coverage because a significant portion of its CRUD logic is tested only against the Firebase emulator in integration tests, which run in a separate configuration.

4.4.2 Client-Side Coverage

File / Directory	% Stmts	% Branch	% Funcs	% Lines
All files	**20.58**	**53.39**	**28.07**	**20.58**
src/pages/				
AdminDashboard.jsx	67.98	44.02	36.11	67.98
Messenger.jsx	73.35	65.27	42.85	73.35
Profile.jsx	45.08	52.52	25.00	45.08
Login.jsx	0	0	0	0
Signup.jsx	0	0	0	0
JobDetail.jsx	0	0	0	0
JobList.jsx	0	0	0	0
JobBid.jsx	0	0	0	0
BidDetail.jsx	0	0	0	0
MyBids.jsx	0	0	0	0
NewJob.jsx	0	0	0	0
Payment.jsx	0	0	0	0
src/components/				
Nav.jsx	78.43	41.66	100	78.43
MapView.jsx	0	0	0	0
AdminRoute.jsx	0	0	0	0
NonAdminRoute.jsx	0	0	0	0
ProtectedApp.jsx	0	0	0	0
src/services/				
config.js	100	100	100	100
socket.js	100	100	100	100
session.js	86.11	60.86	88.23	86.11
api.js	21.35	100	0	21.35
src/hooks/				
useSession.js	62.50	60.00	75.00	62.50
src/util/				
mapHelpers.js	100	100	100	100
locationHelpers.js	100	100	100	100

8 test suites passed, 60 tests passed.

Analysis Client-side tests focus on the components and services that contain the most complex logic. Utility modules (`mapHelpers`, `locationHelpers`, `socket`, `config`) achieve 100% coverage. The service layer (`session.js` at 86%) and key interactive pages (`Messenger` at 73%, `AdminDashboard` at 68%) are well covered. Pages such as `Login`, `Signup`, `JobDetail`, and `Payment` show 0% statement coverage because they are primarily tested through end-to-end user flows in prototype mode rather than isolated component tests. The overall branch coverage of 53.39% reflects strong coverage of conditional logic in the modules that are tested.

5 User Manual

5.1 Introduction

Purpose: OpenBid is a map-first bidding marketplace that connects contractors – individuals or businesses seeking to have work done – with bidders who offer their services. The platform enables job discovery through an interactive map, competitive bidding, secure escrow payments, real-time messaging, and post-job reviews.

Target Audience: Any individual or business seeking to post work or offer professional services on a location-aware platform.

Key Features:

- Visual job discovery on Google Maps
- Competitive bidding and bid management workflow
- Secure Stripe escrow payments with KYC verification
- Real-time in-app messaging between contractors and bidders
- Two-factor authentication via Duo Security
- User portfolios and post-job review system
- Administrative moderation dashboard

5.2 Installation and Setup

5.2.1 Prerequisites

- Node.js v16 or higher
- npm v8 or higher
- Git

5.2.2 Quick Start – Prototype Mode (No External Services Required)

Prototype mode requires no API keys or external accounts. All data is held in memory and resets when the server restarts.

```
# 1. Clone the repository
git clone <repository-url>
cd OpenBid-Map-first-Bidding-Markplace

# 2. Install server dependencies
cd server && npm install

# 3. Install client dependencies
cd ../client && npm install

# 4. Start the backend (prototype mode is the default)
cd ../server
npm run dev
```

```
# 5. In a separate terminal, start the frontend
cd ../client
npm run dev
```

Open `http://localhost:5173` in your browser.

Default Admin Credentials (Prototype Mode):

- Email: `openbidadmin@gmail.com`
- Password: `adminiamadmin`

5.2.3 Production Mode – Firebase and Stripe

For full functionality with persistent data, email verification, KYC, and payments, create the following environment files before starting the application.

server/.env

```
PROTOTYPE=FALSE
PORT=4000
FIREBASE_PROJECT_ID=<your_project_id>
FIREBASE_CLIENT_EMAIL=<your_client_email>
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
FIREBASE_WEB_API_KEY=<your_firebase_web_api_key>
FIREBASE_STORAGE_BUCKET=<your_storage_bucket>
STRIPE_SECRET_KEY_FOR_TESTING=sk_test_<your_stripe_secret_key>
STRIPE_PUBLISHABLE_KEY=pk_test_<your_stripe_publishable_key>
STRIPE_WEBHOOK_SECRET=whsec_<your_webhook_secret>
APP_URL=http://localhost:5173
EMAIL_VERIFICATION_REDIRECT=http://localhost:5173/login
# Optional: Duo 2FA
DUO_CLIENT_ID=
DUO_CLIENT_SECRET=
DUO_API_HOST=
DUO_REDIRECT_URI=
```

client/.env

```
VITE_PROTOTYPE=FALSE
VITE_API_BASE=http://localhost:4000
VITE_GOOGLE_MAPS_API_KEY=<your_google_maps_api_key>
VITE_STRIPE_PUBLISHABLE_KEY=<your_stripe_publishable_key>
```

5.2.4 Running Tests

```
cd server && npm test # Server-side tests
cd client && npm test # Client-side tests
```

5.3 User Interface Guide

5.3.1 Navigation Bar

The navigation bar appears at the top of every page and provides access to all primary sections of the application. The active role (Contractor or Bidder) is displayed prominently and can be changed with the role switch toggle. Admin users see an additional **Admin Dashboard** link.

5.3.2 Map View (Home Page)

The home page displays an interactive Google Maps view with pins representing all open jobs. Clicking any pin opens a popup with the job title, budget, and a link to the full job detail page. The search bar above the map filters visible pins in real time by keyword or location.

5.3.3 Job Detail Page

The job detail page shows the full job description, posted date, budget, and all bids received. Contractors see their own job's bid list and payment action buttons. Bidders see an option to place a bid on jobs they have not yet bid on.

5.3.4 Messenger Page

The Messenger page is divided into a conversation list on the left and an active chat pane on the right. Messages are delivered instantly without refreshing. Selecting a different conversation switches the chat pane.

5.3.5 Profile Page

A user's profile page displays their avatar, bio, portfolio gallery, and a list of reviews received from past jobs. From your own profile, you can edit your bio, upload an avatar, and manage portfolio items.

5.4 Core Features Guide

5.4.1 Creating an Account

1. Navigate to the application and click **Sign Up**.
2. Enter your full name, email address, and a password of at least six characters.
3. In production mode, check your inbox and click the verification link before logging in. In prototype mode, you can log in immediately.
4. Choose an initial role: **Contractor** to post jobs or **Bidder** to apply for jobs.

5.4.2 Logging In and Two-Factor Authentication

1. Enter your email and password on the Login page and click **Log In**.
2. If Duo Security 2FA is enabled on the system, you will be redirected to complete a Duo push notification or enter a passcode.
3. After successful authentication, you are taken to the main job map.

5.4.3 Switching Roles

Click the role indicator in the navigation bar and select the role you wish to switch to. The interface updates immediately to reflect the new role's available actions.

5.4.4 Posting a Job (Contractor)

1. Ensure your active role is set to **Contractor**.
2. Click **Post Job** in the navigation bar.
3. Enter a job title, full description, and your budget.
4. Click on the map to set the job's location pin.
5. Click **Submit**. The job will appear on the map immediately.

5.4.5 Finding and Bidding on Jobs (Bidder)

1. Ensure your active role is set to **Bidder**.
2. Browse the map for job pins or use the search bar to filter by keyword or location.
3. Click a pin and then **View Job** to open the job detail page.
4. Click **Place Bid**, enter your proposed amount and a cover message, then click **Submit Bid**.
5. You can monitor all your submitted bids from the **My Bids** page.

5.4.6 Awarding a Bid and Processing Payment (Contractor)

1. Open one of your job postings from the map or your jobs list.
2. Review the bids received and click **Award Bid** next to your preferred bidder.
3. You will be prompted to fund the escrow. Click **Fund Escrow** and complete the Stripe payment form.
4. Once the job is complete, click **Release Payment** to transfer funds to the bidder. You may need to complete KYC identity verification before your first release.
5. If the job is cancelled, click **Request Refund** to return the escrowed funds.

5.4.7 Messaging

1. Click the **Messages** icon in the navigation bar to open the Messenger page.
2. Select an existing conversation from the list, or start a new conversation from a job or bid detail page.
3. Type your message in the input field at the bottom and press Enter or click **Send**.
4. Messages are delivered in real time. You do not need to refresh the page.
5. To hide a conversation, hover over it in the list and click the hide option. To permanently delete it, use the delete option.

5.4.8 Managing Your Profile and Portfolio

1. Click your name or avatar in the navigation bar to open your profile.
2. Click **Edit Profile** to update your bio.
3. Click the avatar to upload a new profile picture (JPEG or PNG, up to 1 MB).
4. Under the **Portfolio** section, click **Add Item** to add a project showcase with an image, title, and description.

5.4.9 Leaving a Review

1. Navigate to a completed job. Both the contractor and the awarded bidder will see a **Leave a Review** option.
2. Select a star rating from 1 to 5 and write an optional comment.
3. Optionally attach a photo to the review.
4. Submit. The review appears on the reviewee's public profile and their average rating is updated.

5.4.10 Using the Admin Dashboard

1. Log in with an admin account and click **Admin Dashboard** in the navigation bar, or navigate directly to `/admin`.
2. Select the **Users**, **Jobs**, or **Bids** tab to browse the corresponding records.
3. Use the search field at the top of each tab to filter records by keyword.
4. Click the edit icon on any row to open the editor modal. Make changes and click **Save**.
5. Click the delete icon and confirm the deletion dialog to permanently remove a record.

5.5 Troubleshooting and FAQs

Issue	Solution
Cannot log in after sign-up	In production mode, check your inbox for the verification email and click the link before attempting to log in. Check your spam folder if it does not arrive within a minute.
Map does not display any pins	Ensure `VITE_GOOGLE_MAPS_API_KEY` is set in `client/.env`. In prototype mode the map renders with limited tiles but pins will still display.
Fund / Release / Refund button not visible	Payment buttons are visible only to the job's contractor. Ensure you are viewing a job you posted and that the job's current payment status matches the expected action (e.g. Release is only available after funding).

Issue	Solution
Messages not delivering in real time	Confirm the server is running and that the WebSocket connection is not blocked by a firewall, proxy, or browser extension. Try refreshing the page to re-establish the socket connection.
Two-factor authentication not appearing	Duo 2FA is optional and only active when the Duo environment variables are configured in `server/.env`. If they are absent, the login flow skips the second factor.
All data disappears on server restart	This is expected behaviour in prototype mode. Data is held in memory only. Switch to `PROTOTYPE=FALSE` with a configured Firebase project for persistent storage.
KYC verification stuck on "pending"	Stripe Identity verification can take a few minutes to process. Refresh your profile page after completing the identity flow. Ensure the Stripe webhook is correctly configured to point to your server's `/api/kyc/webhook` endpoint.
Admin dashboard shows "Unauthorised"	Only accounts with the `isAdmin` flag set in Firestore can access `/admin`. In prototype mode, use the default admin credentials listed in the Quick Start section above.
Portfolio image upload fails	Ensure the image is a JPEG or PNG and does not exceed 1 MB. Images are automatically resized client-side, but very large or unusual formats may still fail validation.
Review submission returns an error	Ensure the job has been awarded before attempting to leave a review. Each reviewer may submit only one review per job per person – a second submission will be rejected.